

NEU – DS&AI

Data Structures and Algorithms in Python

Lecture 3: Algorithms and Pseudocode Presentations

Faculty of Data Science and Artificial Intelligence

National Economics University

Lecture 3: Table of Contents

- 1 Definition of Algorithms
- 2 Algorithm Representation
- 3 Complexity of Algorithms

By the end of this lecture, students should be able to:

- ① Understand what an algorithm is and identify its essential properties.
- ② Represent algorithms using a flowchart and pseudocode.
- ③ Estimate an algorithm's running time and memory use as functions of the input size.

SECTION 1

Definition of Algorithms

Definition of Algorithms

Algorithm

- A finite, unambiguous sequence of steps for solving a problem.
- A well-defined computational procedure that takes input and produces output.
- A fundamental set of rules used to solve a specific problem.

There is no single formal definition of an “algorithm”



Figure 1: The 9th-century mathematician

Recipe for baking a cake - An example of Algorithm

Recipe: Baking a Cake

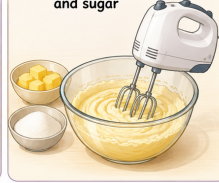
1 Prep your station



2 Mix the dry ingredients



3 Cream the butter and sugar



4 Add the eggs and vanilla



5 Combine wet and dry ingredients

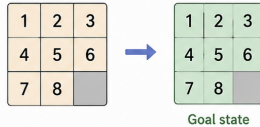


6 Bake the cake



Algorithms Used in Everyday Life

1. Solving puzzles



2. Organizing Bus schedules



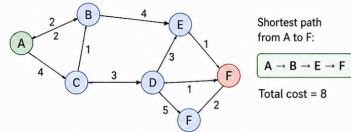
Bus	Route	Departure	Arrival
12	Downtown	08:00	08:45
24	Airport	08:10	08:55
7	University	08:15	09:00
31	City Center	08:20	09:05
...	...	...	...

3. Finding a library book



Title:	Data Structures
Author:	A. Tanenbaum
Call No.:	005.73 T18
Location:	Shelf 3, Row B

4. Find the shortest path



5. Fraud detection

ID	Amount	Location	Time	Flag
1001	\$50	New York	10:01	✓
1002	\$200	New York	10:05	✓
1003	\$9,500	Moscow	10:07	!
1004	\$60	New York	10:10	✓
1005	\$7,200	Tokyo	10:12	!



6. Automated trading



Example algorithm: Solve Quadratic equation

Solving $ax^2 + bx + c = 0$

Input: three integers a, b and c , where $a \neq 0$.

Output: roots of the equation or NONE in case they do not exist.

Algorithm

- 1 Compute $\Delta = b^2 - 4ac$
- 2 If $\Delta < 0$ then there is no root and return "NONE"
- 3 If $\Delta = 0$ then return $-b/(2a)$ as a root
- 4 If $\Delta > 0$ then return $(-b \pm \sqrt{\Delta})/(2a)$ as two different roots

Example of algorithm: calculate Greatest Common Divisor

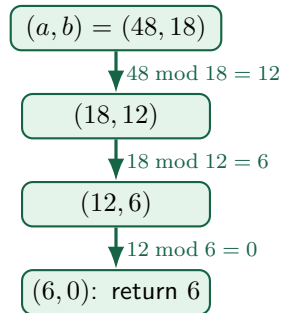
Find $\text{GCD}(a, b)$

Input: two positive integers a and b .

Output: their greatest common divisor.

Euclid's algorithm

- 1 While $b \neq 0$, compute the remainder r of a divided by b .
- 2 Replace a with b and replace b with r .
- 3 When $b = 0$, return a .



The second value reaches 0, so
 $\text{GCD}(48, 18) = 6$.

Example of algorithm: Factorization

Prime factorization

Input: an integer $n > 1$.

Output: the prime factors of n .

Trial-division algorithm

- 1 Start with the possible factor $d = 2$.
- 2 If d divides n , record d and replace n with n/d .
- 3 Otherwise, increase d by 1.
- 4 Repeat until $n = 1$.

Example: factorize 84

Factor tested	Action	Remaining n
2	record 2	42
2	record 2	21
3	record 3	7
4, 5, 6	not divisors	7
7	record 7	1

Result

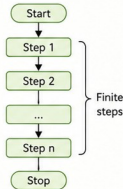
The recorded factors are

2, 3, 7.

Essential properties of an algorithm

1 Finiteness (Termination)

An algorithm must always finish after a finite number of steps.



Example: Finding the maximum in a list will always end.

2 Definiteness (Unambiguity)

Every step must be clear, precise, and unambiguous.

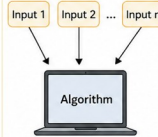
Algorithm Step

1. Read a number
2. If number > 0 do this else do that
3. Print result

Example: Each step is well-defined and leaves no room for doubt.

3 Input specified

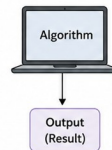
An algorithm must have zero or more well-defined inputs.



Example: Numbers, data, or values are provided to the algorithm.

4 Output specified

An algorithm must produce at least one well-defined output.



Example: The algorithm produces the required result or solution.

5 Effectiveness (Feasibility)

Every step must be basic enough to be carried out in finite time with available resources.



Finite time



Basic operations



Feasible

Example: Steps like addition, comparison, or reading data are simple and executable.

Activity: is this a valid algorithm?

```
FUNCTION FIND_MAX
INPUT: a non-empty list of numbers
OUTPUT: the largest number in the list

    candidate = first_number
    FOR each number p in the list DO
        IF p is much larger than candidate
        THEN
            candidate = p
    IF the list contains 3 numbers THEN
        RETURN candidate
    ELSE
        RETURN first_number
```

Test inputs: [12, 45, 7], [3, 9], and [5, 8, 6, 10].

Validity checklist

- 1 Finiteness
- 2 Definiteness
- 3 Input specified
- 4 Output specified
- 5 Effectiveness

Student task

Identify each violated property, give a reason or counterexample, and suggest how the procedure could be repaired.

Activity: diagnose two procedures

Procedure A: sum of n numbers

```
FUNCTION SUM_TO_N
INPUT: a positive integer  $n$ 
OUTPUT: the sum  $1 + 2 + \dots + n$ 
    total = 0
    i = 1
    WHILE i <= n DO
        total = total + i
    RETURN total
```

Test: $n = 1$ and $n = 3$.

Student task: For each procedure, decide whether it is a valid algorithm. Identify every violated property, justify your answer with a test input, and propose the smallest repair.

Procedure B: calculate an average

```
FUNCTION AVERAGE
INPUT: a list of  $N$  numbers
OUTPUT: the arithmetic mean
    total = 0
    FOR EACH number IN the list DO
        total = total + number
    RETURN total/N
```

Test: $[4, 6]$, $[-2, 2]$, and empty list $[]$.

There may be more than one algorithm for solving a given problem

For example, the GCD can be solved by

- 1 Euclid's algorithm
- 2 Repeated subtraction
- 3 Prime factorization

Not all algorithms are equally efficient

For example, the shortest path problem can be solved by

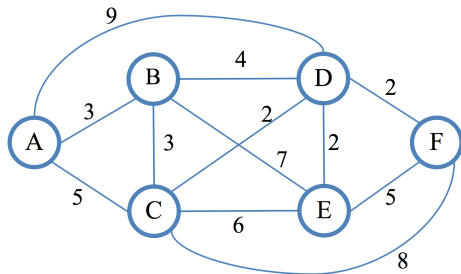
- 1 Brute-force algorithm
- 2 Dijkstra's algorithm
- 3 Bellman-Ford algorithm

Remark

Algorithms may not find the best or optimal solution

For example, the Greedy algorithm for solving shortest path problem

- 1 At each step, select an adjacent vertex with the smallest distance, and continue until the destination is reached.



Find a shortest path from A to F.

- A Greedy algorithm will pick:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E, \quad \text{dis} = 10$$

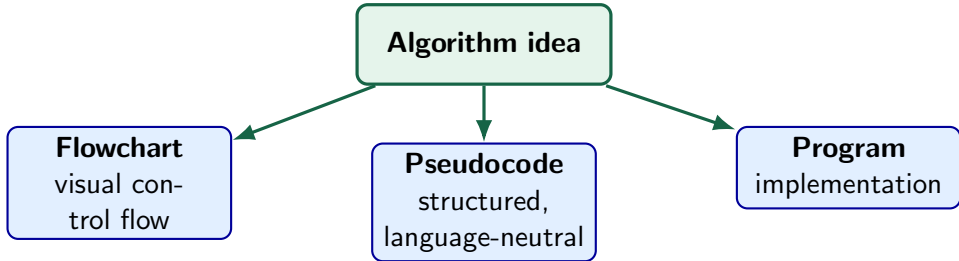
- An optimal path is:

$$A \rightarrow B \rightarrow D \rightarrow F, \quad \text{dis} = 9$$

SECTION 2

Algorithm Representation

Common representations of an algorithm



Flowchart

Definition

A flowchart is a graphical representation of an algorithm. It uses standardized **symbols** connected by **arrows** to show the step-by-step sequence of operations in an algorithm.

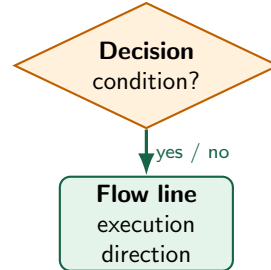
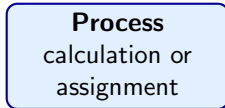
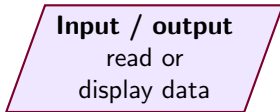
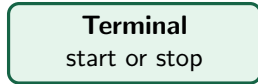
Historical Context

- Flowcharts were first introduced by Frank Gilbreth in 1921 as "Process Charts" to document workflow processes
- They were later refined and popularized in computer science during the 1940s and 1950s as programming became more complex.

Why should flowchart be used?

It provides a visual blueprint that makes complex algorithms easier to understand, analyze, and communicate.

Flowchart symbols



Reading rule

Follow arrows from the start, evaluate each decision, and continue until a terminal node is reached.

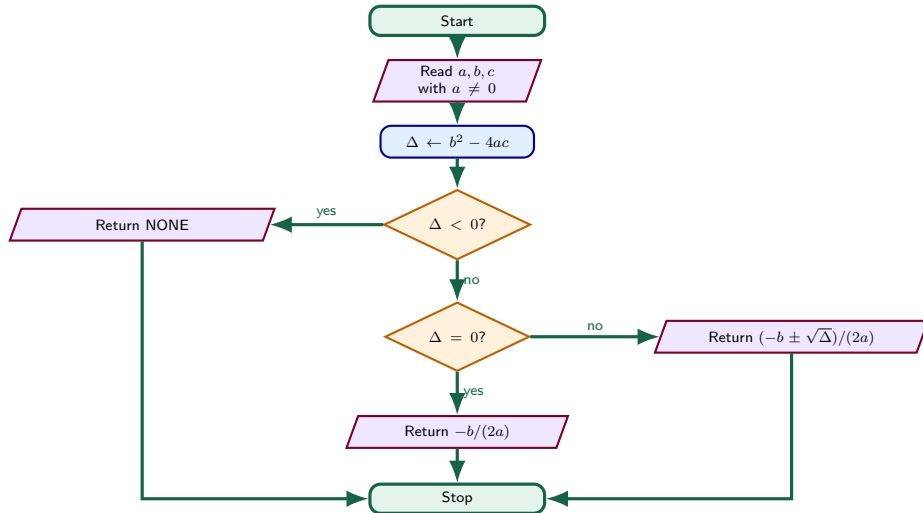
Flowchart Steps

- **Understand the Problem:** Clearly define what needs to be accomplished.
- **Identify Major Steps:** Break down the algorithm into main components.
- **Arrange Steps Logically:** Determine the correct sequence of operations.
- **Draw Symbols:** Use appropriate symbols for each step.
- **Connect with Flowlines:** Show the direction of flow between steps.
- **Review and Test:** Verify the logic by walking through the flowchart.

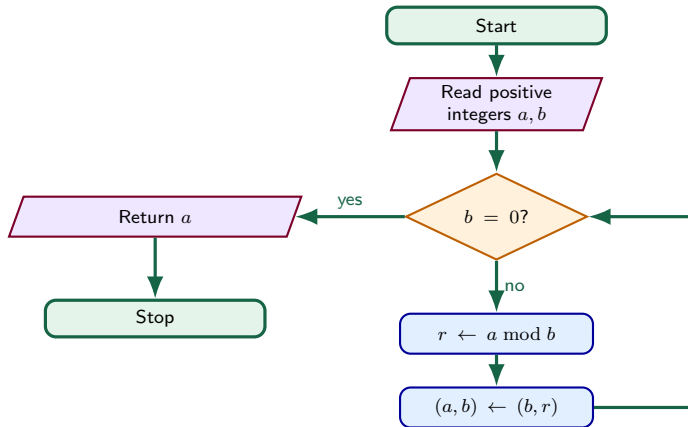
Common Errors to Avoid

- Missing start or end symbols
- Unlabeled decision branches
- Dead ends (paths that don't lead to an end point)
- Ambiguous or unclear text in symbols
- Overly complex diagrams that are difficult to follow

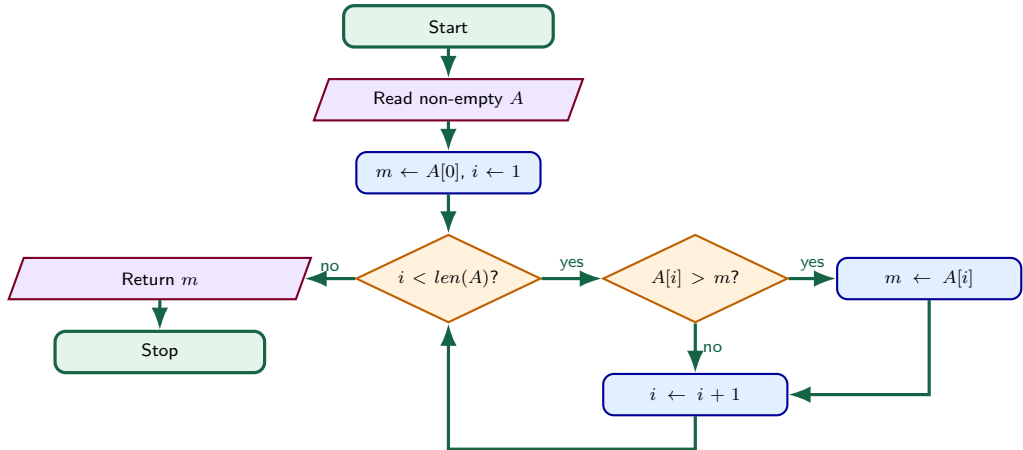
Flowchart: solve a quadratic equation



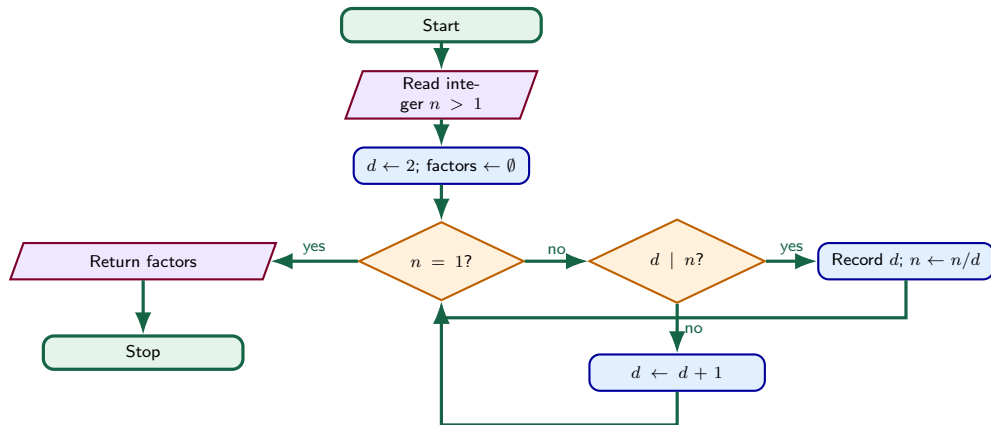
Flowchart: Euclid's GCD algorithm



Flowchart: find max in a list



Flowchart: factorize a number by trial division



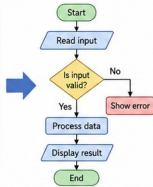
Importance of Flowcharts

1 Visual Clarity

They make algorithm logic visible and easier to comprehend than text-based descriptions.

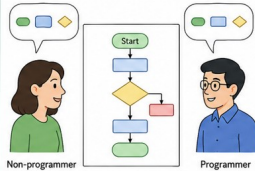
Text Description

1. Read input
2. If input is valid
 - Process data
 - Display result
3. Else
 - Show error
4. End



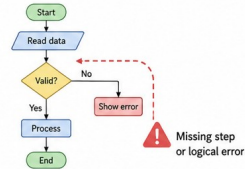
2 Communication Tool

Serve as a universal language that can be understood by programmers and non-programmers alike.



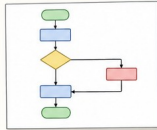
3 Debugging Aid

Help identify logical errors, missing steps, or inefficient sequences before coding begins.



4 Documentation

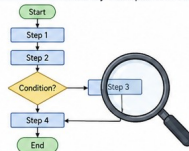
Provide clear documentation of the problem-solving approach for future reference.



- ✓ Problem overview
- ✓ Flowchart
- ✓ Step-by-step logic
- ✓ Notes and assumptions

5 Efficiency Analysis

Allow visual analysis of the algorithm's efficiency and optimization opportunities.



Analysis

- Identify time-consuming steps
- Detect unnecessary loops or steps
- Find opportunities to simplify and optimize

Definition

Pseudocode is a simplified, human-readable representation of an algorithm that uses a mixture of natural language and high level programming language.

Pseudocode is not an actual programming language but rather a structured way to describe the logic of an algorithm without worrying about specific syntax rules.

Pseudocode: An example

ALGORITHM FIND_MAXIMUM

INPUT: a **list** A of m numbers

OUTPUT: the largest number **in** A

```
maximum = A[0]
```

```
FOR i = 1 TO m - 1 DO
```

```
    IF A[i] > maximum THEN
```

```
        maximum = A[i]
```

```
RETURN maximum
```

Language-neutral

The notation communicates the loop and comparison without requiring exact Python syntax.

Implementation-ready

Each pseudocode operation maps naturally to assignment, iteration, and conditionals in a high-level language.

Importance of Pseudo-code

- Easy to Write and Modify: No strict syntax rules to follow
- Focus on Logic: Emphasizes algorithm design over language syntax
- Early Error Detection: Helps identify logical errors before coding
- Better Communication: Serves as excellent documentation for team projects

A comparison

Aspect	Pseudocode	Flowcharts
Ease of Creation	Easy to write and modify	Requires drawing tools
Level of Detail	Can show detailed logic	Shows overall structure well
Modification	Easy to edit text	Difficult to modify complex diagrams
Logic Representation	Good for complex conditions	Excellent for visual learners
Documentation	Good for technical documentation	Better for non-technical audiences

When to Use Each Method

Use Pseudocode When:

- Working with complex logical conditions
- Need quick prototyping and modifications
- Documenting for technical team members
- Preparing for actual coding
- Need quick prototyping and modifications
- Documenting for technical team members
- Preparing for actual coding

Use Flowcharts When:

- Explaining algorithms to non-programmers
- Visualizing overall process flow
- Identifying bottlenecks in processes
- Training purposes

SECTION 3

Complexity of Algorithms

Questions to address

- ① What is Complexity of Algorithms?
- ② Why is it important?
- ③ How can one measure it?

What is Complexity of Algorithms?

The complexity of an algorithm refers to the amount of time and space required to execute the algorithm.



Why is Complexity of Algorithms important?

It allows us to

- 1 Compare the efficiency and performance of different algorithms.
- 2 Choose the most suitable algorithm for a specific problem.
- 3 Optimize our code, reduce execution time, and improve overall program efficiency.
- 4 Predict how our algorithm will scale with larger inputs.



What is Time Complexity of Algorithms?

Definition

Time complexity is the amount of time needed to to run an algorithm.

... *but, how to determine it?*

- ① Measuring seconds on a computer → not a good choice (**why?**)
- ② Counting how many **basic operations** an algorithm performs, supposing that each basic operation takes a fixed amount of time to perform → a good theoretical analysis (**why?**)

Basic Operations in Algorithms

[Image: images/basic_operations.jpg]

Worst-case time complexity

Note: The number of basic operations performed by an algorithm depends on the inputs, and so does algorithm's running time.

Example: Consider a linear search for a number in a given list.

Definition

The worst-case time complexity of an algorithm is the maximum amount of basic operations required for inputs of a given size.

We usually deal with “worst-case time complexity” of algorithms.

The worst-case time complexity of an algorithm for an input of size n is denoted by the function $T(n)$.

Calculating $T(n)$: single loop

```
FUNCTION TOTAL_VALUES
INPUT: a sequence values of length n
OUTPUT: the sum of all values

    result = 0
    FOR EACH value IN values DO
        result = result + value
    RETURN result
```

$$T(n) = 1 + n + 1 = n + 2.$$

Calculating $T(n)$: nested and consecutive loops

```
FOR i = 0 TO n - 1 DO  
  FOR j = 0 TO n - 1 DO  
    PROCESS(i, j)
```

Nested loops multiply

$$T(n) = n \cdot n = n^2.$$

```
FOR EACH item IN a DO  
  PROCESS(item)  
  
FOR EACH item IN b DO  
  PROCESS(item)
```

Consecutive loops add

$$T(n) = n + n = 2n.$$

Logarithmic complexity

```
remaining = n  
WHILE remaining > 1 DO  
    remaining = remaining DIV 2
```

Repeated reduction

Let n be the number of items in the problem. After k iterations, the remaining problem size is approximately $n/2^k$.

Stopping condition

The loop stops when $n/2^k \leq 1$, giving $k \approx \log_2 n$. Therefore, the number of iterations grows logarithmically.

Example: complexity of an IF--THEN--ELSE

```
IF mode = "simple" THEN
  FOR i = 1 TO n DO
    PROCESS(values[i])
  ELSE
    FOR i = 1 TO n DO
      FOR j = 1 TO n DO
        COMPARE(values[i], values[j])
      END IF
    END IF
```

When the condition is true

The condition is checked once and the first loop executes n times:

$$T_{\text{true}}(n) = 1 + n.$$

When the condition is false

The condition is checked once and the nested loops execute $n \cdot n = n^2$ times:

$$T_{\text{false}}(n) = 1 + n^2.$$

Calculating $T(n)$: function calls

```
FUNCTION SUM_VALUES(values, n)
```

```
    total = 0
```

```
    FOR i = 0 TO n - 1 DO
```

```
        total = total + values[i]
```

```
    RETURN total
```

```
FUNCTION DOUBLE_SUM(values, n)
```

```
    first = SUM_VALUES(values, n)
```

```
    second = SUM_VALUES(values, n)
```

```
    RETURN first + second
```

Reuse the called function's cost

Suppose

$$T_{\text{SUM}}(n) = n + 2.$$

DOUBLE_SUM calls it twice and then performs constant extra work.

Combine the costs

$$T_{\text{DOUBLE}}(n) = 2T_{\text{SUM}}(n) + c = 2n + (4 + c),$$

where c represents the fixed cost outside the calls.

Calculating $T(n)$: a changing loop bound

```
FOR i = 1 TO n DO  
  FOR j = 1 TO i DO  
    PROCESS(i, j)
```

The inner loop is not always n

It runs once when $i = 1$, twice when $i = 2$, and n times when $i = n$.

Use a summation

$$T(n) = \sum_{i=1}^n i = 1+2+\dots+n = \frac{n(n+1)}{2}.$$

Nested loops do not always perform exactly n^2 iterations.

Calculating $T(n)$: a changing loop bound

```
FOR i = 1 TO n DO  
  FOR j = 1 TO i DO  
    PROCESS(i, j)
```

The inner loop is not always n

It runs once when $i = 1$, twice when $i = 2$, and n times when $i = n$.

Use a summation

$$T(n) = \sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Nested loops do not always perform exactly n^2 iterations.

Exercise 1: a non-unit step

```
FUNCTION SAMPLE_VALUES(A, n)
  i = 1
  WHILE i <= n DO
    PROCESS(A[i])
    PROCESS(A[i])
    i = i + 3
```

Task

Treat each call to PROCESS as one basic operation.

Determine how many iterations the WHILE loop performs, then derive an exact expression for the number of PROCESS calls.

Exercise 2: a decreasing inner range

```
FUNCTION COMPARE_SUFFIXES(A, n)
  FOR i = 1 TO n DO
    FOR j = i TO n DO
      COMPARE(A[i], A[j])
```

Task

Treat each call to COMPARE as one basic operation.

For a fixed i , count the values of j . Write and evaluate a summation for the total number of comparisons.

Exercise 3: a variable logarithmic loop

```
FUNCTION PREFIX_LEVELS(n)
  FOR i = 1 TO n DO
    step = 1
    WHILE step <= i DO
      PROCESS(i, step)
      step = 2 * step
```

Task

Treat each call to PROCESS as one basic operation.

Count the calls for a fixed i , write a summation over $i = 1, \dots, n$, and determine a tight growth class for $T(n)$.

Exercise 4: three dependent loops

```
FUNCTION NESTED_PREFIX_SUM(A, n)
  total = 0
  FOR i = 1 TO n DO
    FOR j = 1 TO i DO
      FOR k = 1 TO j DO
        total = total + A[k]
  RETURN total
```

Task

Take the addition to `total` as the basic operation.

Write a double summation for its number of executions. Evaluate it to obtain an exact polynomial expression and its tight growth class.

Exercise 5: Sieve of Eratosthenes

```
FUNCTION SIEVE_OF_ERATOSTHENES(n)
  CREATE isPrime[2..n], initially TRUE
  p = 2

  WHILE p * p <= n DO
    IF isPrime[p] = TRUE THEN
      multiple = p * p
      WHILE multiple <= n DO
        isPrime[multiple] = FALSE
        multiple = multiple + p
      p = p + 1

  RETURN all i such that isPrime[i] = TRUE
```

Task

Treat `isPrime[multiple] = FALSE` as the basic operation.

For a fixed prime p , count how many multiples are marked. Then sum this work over the primes $p \leq \sqrt{n}$ and determine the time complexity of the complete algorithm, including initialization.